

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.sql.functions import col, when, regexp_replace
import re # Import re for regex renaming

# Start Spark
spark =
SparkSession.builder.appName("HousingRegression").getOrCreate()

df = spark.read.csv("re.csv", header=True, inferSchema=True)

# Helper function to safely cast to double
def safe_cast_to_double(column):
    # This regex matches integers or floating-point numbers
    # It's a simple check; more complex numbers (e.g., with commas)
    # would need a more elaborate regex
    return when(column.rlike(r"^[0-9]+(\.[0-9]+)?$"),
column.cast("double")).otherwise(None)

# Explicitly cast relevant string columns to double before renaming
and feature assembly
df = df.withColumn("Price", safe_cast_to_double(col("Price")))
df = df.withColumn("Total_Area",
safe_cast_to_double(col("Total_Area")))
df = df.withColumn("Price_per_SQFT",
safe_cast_to_double(col("Price_per_SQFT")))
df = df.withColumn("Baths", safe_cast_to_double(col("Baths")))
# Convert 'Yes'/'No' in Balcony to 1.0/0.0 before casting to double
df = df.withColumn("Balcony",
    when(col("Balcony") == "Yes", 1.0)
    .when(col("Balcony") == "No", 0.0)
    .otherwise(None)
    .cast("double")
)

print("Schema after casting numeric columns:")
df.printSchema()

# Rename columns to replace spaces and periods with a SINGLE
underscore for VectorAssembler compatibility
original_columns = df.columns
new_columns = [re.sub(r'[ .]+', '_', col_name) for col_name in
original_columns] # Use regex to replace ONE OR MORE spaces and dots
with single underscore
for original, new in zip(original_columns, new_columns):
    df = df.withColumnRenamed(original, new)
```

```

# Select features using the renamed column names that are actually
present and cast to numeric
assembler = VectorAssembler(
    inputCols=[
        "Total_Area",
        "Price_per_SQFT",
        "Baths",
        "Balcony"
    ],
    outputCol="features",
    handleInvalid="skip" # Skip rows with null values
)

data = assembler.transform(df)

# Remove rows with nulls in the 'features' column (if any remained
after handleInvalid="skip")
# Also remove rows where 'Price' (the label column) is null
data = data.dropna(subset=["features", "Price"])

# Optional scaling
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaled_features"
)

schaler_model = scaler.fit(data)
data = schaler_model.transform(data)

# Train-test split
train_data, test_data = data.randomSplit([0.8, 0.2])

# Model
lr = LinearRegression(
    featuresCol="scaled_features",
    labelCol="Price" # 'Price' is now cast to double and used as label
)

# Check if test_data is empty before proceeding with predictions and
evaluation
if test_data.count() == 0:
    print("Test data is empty. Cannot evaluate the model.")
    # Optionally, you could exit or set RMSE/R2 to default values
    rmse = float('nan')
    r2 = float('nan')
else:
    model = lr.fit(train_data)

# Predictions

```

```

predictions = model.transform(test_data)

predictions.select("Price", "prediction").show(10)

# Evaluation
rmse = RegressionEvaluator(
    labelCol="Price",
    predictionCol="prediction",
    metricName="rmse"
).evaluate(predictions)

r2 = RegressionEvaluator(
    labelCol="Price",
    predictionCol="prediction",
    metricName="r2"
).evaluate(predictions)

print("RMSE:", rmse)
print("R2 Score:", r2)

# Save model
# Only save if model was actually trained (i.e., test_data was not empty)
if 'model' in locals():
    model.write().overwrite().save("house_price_model")
else:
    print("Model was not trained or saved due to empty test data.")

```

Schema after casting numeric columns:

```

root
|-- Name: string (nullable = true)
|-- Property Title: string (nullable = true)
|-- Price: double (nullable = true)
|-- Location: string (nullable = true)
|-- Total_Area: double (nullable = true)
|-- Price_per_SQFT: double (nullable = true)
|-- Description: string (nullable = true)
|-- Baths: double (nullable = true)
|-- Balcony: double (nullable = true)

```

Test data is empty. Cannot evaluate the model.

RMSE: nan

R2 Score: nan

Make a Prediction with New User Input

Now, let's allow you to input values for the features and get a price prediction from the trained model.

```

import pandas as pd
from pyspark.sql.types import StructType, StructField, DoubleType

print("Enter values for prediction:")

try:
    total_area = float(input("Total Area (sqft): "))
    price_per_sqft = float(input("Price per SQFT: "))
    baths = float(input("Number of Baths: "))

    balcony_input = input("Balcony (Yes/No): ")
    balcony = 1.0 if balcony_input.lower() == 'yes' else (0.0 if
    balcony_input.lower() == 'no' else None)

    # Create a Pandas DataFrame for the single user input row
    user_data_pd = pd.DataFrame([{
        "Total_Area": total_area,
        "Price_per_SQFT": price_per_sqft,
        "Baths": baths,
        "Balcony": balcony
    }])

    # Define the schema to ensure correct types in Spark DataFrame
    schema = StructType([
        StructField("Total_Area", DoubleType(), True),
        StructField("Price_per_SQFT", DoubleType(), True),
        StructField("Baths", DoubleType(), True),
        StructField("Balcony", DoubleType(), True)
    ])

    # Convert to Spark DataFrame
    user_df = spark.createDataFrame(user_data_pd, schema=schema)

    # Display the user input DataFrame
    print("\nUser Input Data:")
    user_df.show()

    # Apply VectorAssembler
    user_features = assembler.transform(user_df)

    # Apply StandardScaler
    scaled_user_features = schaler_model.transform(user_features)

    # Make prediction if model exists
    if 'model' in locals():
        prediction_result = model.transform(scaled_user_features)
        predicted_price =
prediction_result.select("prediction").collect()[0][0]
        print(f"\nPredicted Price: {predicted_price:.2f}")
    else:
        print("\nModel not found. Please train the model first by

```

```
running the previous cells successfully.")
    print("The previous run reported: 'Model was not trained or
saved due to empty test data.'")
```

```
except ValueError:
    print("Invalid input. Please ensure you enter numeric values for
Total Area, Price per SQFT, and Baths.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

Enter values for prediction:

Total Area (sqft): 1000

Price per SQFT: 8000

Number of Baths: 2

Balcony (Yes/No): yes

User Input Data:

+	-----+	-----+	-----+	-----+
	Total_Area	Price_per_SQFT	Baths	Balcony
+	-----+	-----+	-----+	-----+
	1000.0	8000.0	2.0	1.0
+	-----+	-----+	-----+	-----+

Predicted Price: -0.25

CT2 - Sem2 Reg: RA2512044015105 ; Name: Kannan S

Q2. Classification Model with Spark MLlib Use Case: Predicting customer churn in a telecom dataset. Procedure: Load telecom customer dataset into Spark. Split into training and test sets. Train a classification model (e.g., Logistic Regression or Decision Tree) using Spark MLlib. Evaluate accuracy, precision, recall, and F1 score. Deliverables: Trained classification model. Performance metrics report.

```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler, StringIndexer
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
from pyspark.sql.functions import col, expr # Import expr for try_cast

# Start Spark
spark = SparkSession.builder.appName("ChurnPrediction").getOrCreate()

# Load dataset
df = spark.read.csv("telecom.csv", header=True, inferSchema=True)

# Convert 'TotalCharges' to numeric, handling potential errors by
# coercing to null and then filling with 0
df = df.withColumn("TotalCharges", expr("try_cast(TotalCharges AS
DOUBLE)")) # Use try_cast
df = df.na.fill(0, subset=["TotalCharges"])

# Convert target column
label_indexer = StringIndexer(inputCol="Churn", outputCol="label")
df = label_indexer.fit(df).transform(df)

# Convert categorical columns
gender_indexer = StringIndexer(inputCol="gender",
outputCol="gender_index")
df = gender_indexer.fit(df).transform(df)

contract_indexer = StringIndexer(inputCol="Contract",
outputCol="contract_index")
df = contract_indexer.fit(df).transform(df)

# Feature vector
assembler = VectorAssembler(
    inputCols=[
        "tenure",
        "MonthlyCharges",
        "TotalCharges",
        "gender_index",
        "contract_index"
    ],
    outputCol="features"
)
```

```
data = assembler.transform(df)

# Split data
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)

# Train model
lr = LogisticRegression(featuresCol="features", labelCol="label")
model = lr.fit(train_data)

# Prediction
predictions = model.transform(test_data)

# Evaluation
evaluator = MulticlassClassificationEvaluator(labelCol="label",
predictionCol="prediction")

accuracy = evaluator.setMetricName("accuracy").evaluate(predictions)
precision =
evaluator.setMetricName("weightedPrecision").evaluate(predictions)
recall =
evaluator.setMetricName("weightedRecall").evaluate(predictions)
f1 = evaluator.setMetricName("f1").evaluate(predictions)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

# Save model
model.save("churn_model")

Accuracy: 0.7858736059479554
Precision: 0.7734249495928949
Recall: 0.7858736059479554
F1 Score: 0.7745976969806873
```